Localization with Acceleration

Author: Ella Ilan

Date: 2025-01-10 to 2025-01-17

Relevant Links:

- Project DR0

The goal of this project was to investigate if we can localize data from FSGP 2025 effectively. We're going to do so in this notebook by investigating acceleration data. We're going to create a predicted model of the radius of curvature we expect to see on the race track and compare it to a calculated one using our accelerometers.

Imports

Import any packages or dependencies for this project. Uncomment or delete these lines as needed, and add other dependencies

In [2]:
```
import numpy as np
from data_tools import query, TimeSeries
from datetime import datetime
import pytz
import pandas as pd
import os
import sys
import json
import requests
from tqdm import tqdm
from dotenv import load_dotenv
```

Data Acquisition

This code riffs off of Jonah's acceleration analysis. I tried working with 2024 data and our data tools, then realized we don't have acceleration sensors on the car from that time, so my query returned empty! This is 2025 data from our third and fourth lap. I query it from our database.

In [3]:
```
client = query.DBClient()
#convert local to UTC even though the local time is wrong, stolen from Jonah
hour_fix = 7
#chose the best lap with no pitting from the third day of competition
#manually creating date time objects using lap times from spreadsheet
#13:28:36
```

```
lap_start_3_3 = datetime(2025, 7, 5, 11 + hour_fix, 28, 36, tzinfo=pytz.UTC)
#13:34:32
lap_stop_3_3 = datetime(2025, 7, 5, 11 + hour_fix, 34, 32, tzinfo=pytz.UTC)

lap_start_3_4 = datetime(2025, 7, 5, 11 + hour_fix, 34, 32, tzinfo=pytz.UTC)
#13:40:43
lap_stop_3_4 = datetime(2025, 7, 5, 11 + hour_fix, 40, 43, tzinfo=pytz.UTC)


lap_start_3_4 = datetime(2025, 7, 5, 11 + hour_fix, 34, 32, tzinfo=pytz.UTC)
#13:40:43
lap_stop_3_4 = datetime(2025, 7, 5, 11 + hour_fix, 40, 43, tzinfo=pytz.UTC)


print(lap_start_3_3)
print(lap_stop_3_3)

fields = ["Acceleration_X", "Acceleration_Y", "Acceleration_Z"]

# convert milli-Gs to m/s^2
G = 9.81
print("Collecting Lap Data")
raw_track_data = {}
for field in fields:
    print(f"    Queried {field}!")
    raw_track_data[field] = client.query_time_series(lap_start_3_3, lap_stop_3_3, f

#covert from km/hr to m/s
raw_track_data["MotorRotatingSpeed"] = client.query_time_series(lap_start_3_3, lap_
print(f"    Queried MotorRotatingSpeed!")


x, y, z, motor = TimeSeries.align(raw_track_data["Acceleration_X"], raw_track_data[
```

```
2025-07-05 18:28:36+00:00
2025-07-05 18:34:32+00:00
Collecting Lap Data
    Queried Acceleration_X!
    Queried Acceleration_Y!
    Queried Acceleration_Z!
    Queried MotorRotatingSpeed!
```

Plotting our expected Radius of Curvature

Using the Google Earth data, we can find expected radius of curvature. This is a list of coordinates pulled off of our Google Earth map of the FSGP track.

In [17]:
```
#These are the coordinates of the track, stolen from Miguel

reverse_coords = [
    [ 37.0011529 , -86.36837867],
    [ 37.00122817, -86.3682181 ],
    [ 37.00133071, -86.36801267],
    [ 37.00143614, -86.36779264],
```

```
    [ 37.00152389, -86.3675912 ],
    [ 37.00160574, -86.36740819],
    [ 37.00167596, -86.36725066],
    [ 37.00175285, -86.36709064],
    [ 37.00183166, -86.36691875],
    [ 37.00192538, -86.36670617],
    [ 37.00200136, -86.36653034],
    [ 37.00208623, -86.36635086],
    [ 37.00215644, -86.36619701],
    [ 37.00222549, -86.36603626],
    [ 37.00229839, -86.3658645 ],
    [ 37.00237732, -86.36569622],
    [ 37.00245038, -86.36553914],
    [ 37.00252912, -86.36537128],
    [ 37.00259904, -86.36521818],
    [ 37.00266755, -86.36507091],
    [ 37.00274639, -86.36490341],
    [ 37.00283342, -86.36471029],
    [ 37.00291248, -86.36454704],
    [ 37.00298517, -86.36439075],
    [ 37.0030636 , -86.36423803],
    [ 37.00313338, -86.36408574],
    [ 37.00320937, -86.36393701],
    [ 37.00330797, -86.36377724],
    [ 37.00343811, -86.36368662],
    [ 37.00357758, -86.36365019],
    [ 37.00372489, -86.36360692],
    [ 37.00388711, -86.36356354],
    [ 37.00405472, -86.36352621],
    [ 37.00423763, -86.36348621],
    [ 37.00437129, -86.36338074],
    [ 37.00448184, -86.36323899],
    [ 37.00457515, -86.36307953],
    [ 37.0047012 , -86.36286956],
    [ 37.00486024, -86.36273924],
    [ 37.00505061, -86.36270756],
    [ 37.00527945, -86.36272947],
    [ 37.00548802, -86.36263566],
    [ 37.00565341, -86.36245496],
    [ 37.00573513, -86.3621647 ],
    [ 37.00568611, -86.36182869],
    [ 37.00548782, -86.36157939],
    [ 37.0052881 , -86.36149696],
    [ 37.00511652, -86.36149669],
    [ 37.0049746 , -86.36158761],
    [ 37.00485989, -86.3616726 ],
    [ 37.00469955, -86.3617696 ],
    [ 37.00451492, -86.36178471],
    [ 37.00435852, -86.36173599],
    [ 37.00419576, -86.36162316],
    [ 37.00409127, -86.36146652],
    [ 37.00404463, -86.3612301 ],
    [ 37.00407232, -86.36097832],
    [ 37.00415922, -86.36078578],
    [ 37.00426711, -86.36066944],
    [ 37.00439509, -86.36060407],
```

```
    [ 37.00452844, -86.36057503],
    [ 37.00466778, -86.36054604],
    [ 37.004833  , -86.36050987],
    [ 37.00499495, -86.36047743],
    [ 37.00514229, -86.36044484],
    [ 37.00524902, -86.36041601],
    [ 37.00541074, -86.36037856],
    [ 37.00558676, -86.36034575],
    [ 37.00578957, -86.36038183],
    [ 37.00596102, -86.36045445],
    [ 37.00607154, -86.36065091],
    [ 37.0061651 , -86.36090615],
    [ 37.00626234, -86.3611639 ],
    [ 37.00637312, -86.36147776],
    [ 37.00642557, -86.36179897],
    [ 37.00644459, -86.36216748],
    [ 37.00637929, -86.36250433],
    [ 37.00629836, -86.36273906],
    [ 37.00622229, -86.36291634],
    [ 37.00611158, -86.36309997],
    [ 37.00600738, -86.36323379],
    [ 37.00589169, -86.3633461 ],
    [ 37.00576471, -86.36344689],
    [ 37.00564905, -86.36353398],
    [ 37.00551878, -86.36359932],
    [ 37.00537978, -86.36365448],
    [ 37.00525188, -86.36370605],
    [ 37.0051236 , -86.36376501],
    [ 37.00497727, -86.36382802],
    [ 37.00484855, -86.36391978],
    [ 37.00471266, -86.36403361],
    [ 37.0045783 , -86.36422396],
    [ 37.00447907, -86.36439954],
    [ 37.00435695, -86.36453807],
    [ 37.00424813, -86.36465022],
    [ 37.00411061, -86.36480228],
    [ 37.00397977, -86.36492984],
    [ 37.0038682 , -86.36504946],
    [ 37.00376295, -86.36516126],
    [ 37.00365436, -86.36527329],
    [ 37.00354759, -86.36539144],
    [ 37.00341675, -86.36552332],
    [ 37.00329206, -86.3656715 ],
    [ 37.00316095, -86.36581567],
    [ 37.00303256, -86.36594441],
    [ 37.00291203, -86.36607157],
    [ 37.00278014, -86.36622802],
    [ 37.00269333, -86.36641303],
    [ 37.00260606, -86.36659437],
    [ 37.00252845, -86.36675984],
    [ 37.00243487, -86.36695271],
    [ 37.00234795, -86.36716654],
    [ 37.00226102, -86.36735999],
    [ 37.00218046, -86.36754144],
    [ 37.00210026, -86.36771854],
    [ 37.00202358, -86.36787901],
```

```
    [ 37.00194406, -86.36806693],
    [ 37.00185835, -86.36825428],
    [ 37.00177578, -86.36843604],
    [ 37.00169902, -86.36860048],
    [ 37.0016732 , -86.3687928 ],
    [ 37.001679  , -86.3689974 ],
    [ 37.00168488, -86.36919051],
    [ 37.00169122, -86.36938766],
    [ 37.00169469, -86.3695562 ],
    [ 37.00170118, -86.36974238],
    [ 37.00171055, -86.36993991],
    [ 37.00167198, -86.37010864],
    [ 37.00158558, -86.37021273],
    [ 37.00148934, -86.37030898],
    [ 37.00139295, -86.37041757],
    [ 37.00129011, -86.37051816],
    [ 37.00120286, -86.37061913],
    [ 37.00112253, -86.37073184],
    [ 37.00106144, -86.37085264],
    [ 37.00099678, -86.37100197],
    [ 37.00092255, -86.37116696],
    [ 37.00083249, -86.37137665],
    [ 37.00073506, -86.37159438],
    [ 37.00065426, -86.37177611],
    [ 37.00057398, -86.37196168],
    [ 37.00060097, -86.37214635],
    [ 37.00074272, -86.37221467],
    [ 37.0009059 , -86.37222018],
    [ 37.00107657, -86.37218846],
    [ 37.00122503, -86.37211976],
    [ 37.00133803, -86.37204677],
    [ 37.00145467, -86.37192099],
    [ 37.00156457, -86.37178347],
    [ 37.00167114, -86.37162599],
    [ 37.00176434, -86.37145299],
    [ 37.00183833, -86.37126821],
    [ 37.00188646, -86.37112396],
    [ 37.0019441 , -86.37095574],
    [ 37.00197296, -86.370787  ],
    [ 37.00202945, -86.37067751],
    [ 37.00209668, -86.37048098],
    [ 37.00218305, -86.37031655],
    [ 37.00228558, -86.37016005],
    [ 37.00238818, -86.37002764],
    [ 37.00249377, -86.36991909],
    [ 37.00259639, -86.36983868],
    [ 37.00270864, -86.36976266],
    [ 37.00281411, -86.3697033 ],
    [ 37.00292264, -86.36964769],
    [ 37.00303847, -86.369599  ],
    [ 37.00316942, -86.36952703],
    [ 37.00332578, -86.36939126],
    [ 37.00342743, -86.36921198],
    [ 37.00346211, -86.36901195],
    [ 37.00343231, -86.36879509],
    [ 37.00336736, -86.36861758],
```

```
    [ 37.00327983, -86.36847602],
    [ 37.00316589, -86.36828932],
    [ 37.00305696, -86.36810259],
    [ 37.00296937, -86.36793662],
    [ 37.00293802, -86.36773416],
    [ 37.00295792, -86.36753614],
    [ 37.00301643, -86.36739089],
    [ 37.00307192, -86.36724611],
    [ 37.00312788, -86.36711056],
    [ 37.00320856, -86.36698638],
    [ 37.00331523, -86.36684361],
    [ 37.00340519, -86.36672019],
    [ 37.00350264, -86.36658791],
    [ 37.00361135, -86.36649982],
    [ 37.00374245, -86.36645177],
    [ 37.00387034, -86.3664758 ],
    [ 37.00395668, -86.36655587],
    [ 37.00402368, -86.36670753],
    [ 37.00407807, -86.36686779],
    [ 37.00412607, -86.36701603],
    [ 37.00418686, -86.36718831],
    [ 37.00427315, -86.36742386],
    [ 37.00436296, -86.3676927 ],
    [ 37.00442144, -86.36790543],
    [ 37.00445858, -86.36815496],
    [ 37.00447715, -86.36847342],
    [ 37.00444541, -86.36882644],
    [ 37.00434238, -86.36911386],
    [ 37.00425654, -86.36927291],
    [ 37.00418289, -86.36939303],
    [ 37.00410293, -86.36950912],
    [ 37.00399731, -86.36962489],
    [ 37.00390147, -86.36973254],
    [ 37.00379293, -86.36984054],
    [ 37.00369701, -86.36994104],
    [ 37.00359474, -86.37004514],
    [ 37.0034838 , -86.3701598 ],
    [ 37.00338477, -86.37024376],
    [ 37.00327609, -86.37032788],
    [ 37.00318982, -86.37040789],
    [ 37.00306205, -86.37049992],
    [ 37.00294389, -86.37059192],
    [ 37.00283177, -86.37070006],
    [ 37.00272312, -86.37083897],
    [ 37.00264003, -86.37094293],
    [ 37.00255683, -86.37103895],
    [ 37.00249914, -86.37115127],
    [ 37.00243497, -86.37126771],
    [ 37.00236746, -86.3713964 ],
    [ 37.00230001, -86.37152106],
    [ 37.00223561, -86.37164596],
    [ 37.00217132, -86.37175865],
    [ 37.0021135 , -86.37187932],
    [ 37.00204601, -86.37200003],
    [ 37.00196791, -86.37211434],
    [ 37.00188761, -86.3722312 ],
```

```
    [ 37.00180726, -86.37233202],
    [ 37.00173651, -86.37241676],
    [ 37.00165284, -86.37250555],
    [ 37.00155067, -86.3726109 ],
    [ 37.00144762, -86.37271211],
    [ 37.00136704, -86.37279282],
    [ 37.00126713, -86.37288565],
    [ 37.00115123, -86.37299929],
    [ 37.00103518, -86.37310829],
    [ 37.0009191 , -86.37321738],
    [ 37.00082561, -86.37329809],
    [ 37.00072863, -86.37339546],
    [ 37.00061364, -86.37350099],
    [ 37.00051698, -86.37360214],
    [ 37.00042667, -86.37368311],
    [ 37.00032338, -86.37378434],
    [ 37.00022342, -86.37387665],
    [ 37.00012018, -86.3739737 ],
    [ 37.00002051, -86.37405916],
    [ 36.99991404, -86.37415222],
    [ 36.99981076, -86.37423248],
    [ 36.99970432, -86.37430523],
    [ 36.99958169, -86.37432571],
    [ 36.99946928, -86.37430399],
    [ 36.99934964, -86.37428072],
    [ 36.99922365, -86.37425686],
    [ 36.99908793, -86.37422888],
    [ 36.99899069, -86.37413638],
    [ 36.99896465, -86.37397888],
    [ 36.9989808 , -86.37380904],
    [ 36.99900664, -86.37362303],
    [ 36.99904589, -86.37342887],
    [ 36.9990885 , -86.37323096],
    [ 36.99912743, -86.3730292 ],
    [ 36.99916641, -86.37284378],
    [ 36.99921127, -86.37261758],
    [ 36.99924349, -86.37245211],
    [ 36.99932082, -86.37230251],
    [ 36.99932082, -86.37230251],
    [ 36.99940337, -86.37214797],
    [ 36.99948814, -86.37199725],
    [ 36.99958754, -86.37189043],
    [ 36.99970528, -86.37178738],
    [ 36.99982625, -86.37169183],
    [ 36.99997073, -86.37159181],
    [ 37.00011471, -86.3714889 ],
    [ 37.00027357, -86.37138907],
    [ 37.0003852 , -86.37123775],
    [ 37.00042033, -86.37099884],
    [ 37.00038811, -86.3707825 ],
    [ 37.00031195, -86.37062837],
    [ 37.00026491, -86.37041498],
    [ 37.00030254, -86.37020247],
    [ 37.00038472, -86.3700261 ],
    [ 37.0004699 , -86.36984594],
    [ 37.00056383, -86.36964375],
```

```
    [ 37.00064015, -86.3694857 ],
    [ 37.00070993, -86.36934481],
    [ 37.0008098 , -86.36912411],
    [ 37.00090662, -86.36891836],
    [ 37.00098578, -86.36874579],
    [ 37.00107373, -86.36854755]
  ]

coords = reverse_coords[::-1] # coordinates in the correct order; starting coordina
```

Using some math, we can use some given number of nearest points to calculate curvature given points and their neighbours.

In [18]:
```
import math

def calculate_circular_track_curvature(coords, step=1):
    """
    Calculates curvature for a CLOSED LOOP (circular) race track.

    :param coords: List of tuples [(lat, lon), ...] assumed to be in order.
    :param step: The smoothing stride (1=sensitive, 5=smooth).
    :return: List of radii in meters.
    """

    if not coords:
        return []

    # 1. Convert Lat/Lon to Local X/Y (Meters)
    # We use the first point as the reference center for projection
    center_lat = coords[0][0]

    # Calculate conversion factors based on the track's latitude
    meters_per_deg_lat = 111132.954 - 559.822 * math.cos(2 * math.radians(center_la
    meters_per_deg_lon = 111412.84 * math.cos(math.radians(center_lat))

    xy_points = []
    for lat, lon in coords:
        y = lat * meters_per_deg_lat
        x = lon * meters_per_deg_lon
        xy_points.append((x, y))

    radii = []
    n = len(xy_points)

    # 2. Calculate Radius with Wrap-Around Indexing
    for i in range(n):
        # Use modulo (%) to wrap around the start/finish line
        # If i=0 and step=1, the "previous" point becomes the last point in the lis
        p1 = xy_points[(i - step) % n]
        p2 = xy_points[i]
        p3 = xy_points[(i + step) % n]

        # Calculate side lengths (Euclidean distance)
        a = math.dist(p1, p2)
        b = math.dist(p2, p3)
```

```
        c = math.dist(p3, p1)

        # Shoelace formula for Area of the triangle
        area = 0.5 * (p1[0]*(p2[1] - p3[1]) +
                        p2[0]*(p3[1] - p1[1]) +
                        p3[0]*(p1[1] - p2[1]))

        # Calculate Radius (R = abc / 4A)
        if  -1e-6 < area < 1e-6:
            radii.append(0) # Straight line
        else:
            R = (a * b * c) / (4 * area)
            #if R > 10000:
              #  R = 0
            radii.append(1/R)


    return radii
```

This function gives us a list of radii of curvature.

In [19]:
```
radius_of_curvature = calculate_circular_track_curvature(coords, step=2)
print(radius_of_curvature)
```

[-0.0004737137265891718, 0.000243934571673653, 0.00024396305804477673, 9.81556012766
9603e-05, 0.0006777073448286618, 8.525624908714688e-05, -0.0005287046922819839, -7.9
93123254774308e-05, -0.004229839934190933, -0.01473267160975897, -0.0205344550012840
32, -0.008792581336408641, 0.010564927478842797, 0.02091709903912689, 0.022848948681
27058, 0.0164384483443901, 0.003405124980447767, -0.0011314379212440857, -0.00217761
38636877018, -0.003883199526559917, -0.00810583677357186, -0.009800027549562058, -0.
005949014706963355, -0.0011084087775854359, -0.007570867465304938, -0.01275063609893
6754, -0.004689004078527874, 0.00014565047484554406, 1.0548041730424146e-05, 0.00024
750131463062197, -0.0010882289413032922, -0.003190982406647334, -0.0077635250092525
5, -0.023504580177961275, -0.037778632963728534, -0.030082700503327297, -0.008632498
535673303, -0.00011852006756935844, -0.005525830495296073, -0.017624651043452653, -
0.019876092618213385, -0.009837614649622784, -0.0028316593144832435, -0.001424466030
2293083, -0.0012414175595190395, -0.0009716436497567395, -0.00017180267158149826, -
0.00040091258230592056, -0.0006287894322261986, 0.00030175568033500905, 0.0007466116
898320741, 0.0009086944123143459, -7.976282135183099e-05, -0.000910363418973391, -0.
00028003324488079707, -2.842466000279187e-05, -0.0006020549440885876, -0.00081586894
84518098, -0.0009817017007399801, -0.0020773813830796952, -0.0031878066774360812, -
0.003781420746132173, -0.0033978407747367865, -0.0032664020168333293, -0.00503247727
0210297, -0.0031074413672654267, 0.0007642824355742878, 0.00017179628289867009, -0.0
0029293189254070467, 0.00028993808977776254, -1.267387833410775e-07, 0.0043685200972
19849, 0.008657679507706002, 0.0028206949471427946, 0.0012263741116137186, 0.0059946
587406795385, 0.005887328616936731, 0.0012165128215119812, -0.001842168106361101, -
0.00011016237030813625, -0.0019912310966491895, -0.003881997974901288, -0.0014716042
072770881, 0.00010029168072394837, -0.00036906839340482267, -0.0013982139573605898,
-0.002856728691791349, -0.005698557628562199, -0.005540629269201604, -0.005435035305
027754, -0.008048793735389304, -0.008726094507553784, -0.006849157444229499, -0.0053
98858125988475, -0.005047834718907637, -0.003457148626658626, -0.001306874821895662
8, 0.0002560904891602207, 0.000869477664959866, -0.001663752836490112, -0.0105031106
39071694, -0.025888542854512978, -0.033082236158225475, -0.02838958361245904, -0.021
16003214687586, -0.013330342310370212, -0.0037761676126333962, -0.000690863148962197
4, -0.005467139045829984, -0.008684355545704958, -0.004426909864990946, -0.006125360
6458302, -0.015105163110749541, -0.019324051629949955, -0.0152671135228626, -0.00668
01702175657025, -0.0012270607268325185, 0.0027484661860925443, 0.00981358387121883,
0.015540155567884774, 0.018951994447085268, 0.01973824581069902, 0.01662823314267099
6, 0.011464271924939621, 0.005309396700892424, -0.0011983832176628707, -0.0039446286
92720242, -0.004546433158410676, -0.005986846250153897, -0.007415942337036704, -0.00
6883800139679239, -0.005652142691519473, -0.006684468543310883, -0.00503757317093896
2, -0.003895087108057111, -0.005067008978174203, 0.0012018472129183555, 0.0038782546
766402406, 0.003823237828998271, 0.00613925224859106, 0.005883159895337009, 0.004899
804580620769, 0.006680945368493163, 0.010260697194134673, 0.010587226633719181, 0.01
0041857145876718, 0.012263338676021885, 0.027224927832705876, 0.041331102265187276,
0.03369705094245718, 0.008865032168924586, 2.4676526473108674e-06, -0.00018708974385
230824, 8.195876387953191e-05, 0.0007778386765653965, 0.004936657143143716, 0.009107
63118095022, 0.008146734892396324, 0.0034841430631166806, -5.076450561993878e-05, -
0.0007741419293388207, -0.012275387136677009, -0.024516612727282724, -0.018107383033
022596, -0.004166578191622187, 0.0005412171496625496, -0.00012145337000818752, -9.31
8765555489281e-05, 0.0029034231630538994, 0.010954216532269427, 0.01281368664144319
5, 0.004824419905751995, -0.0005134095922403875, -0.00019519417486944996, 0.00063493
55339840349, 3.5174952512363667e-07, -0.00046481721711743803, -0.000532572385767836
5, -7.053396411596431e-05, 0.0012314313934618654, 0.0007681911929349385, -6.28366377
0930208e-05, 0.0037102805679199696, 0.007973146126542296, 0.005544196998699033 6, 0.0
008688176526792117, -0.0014105380931843184, -0.0005411680045708521, 0.00112182128994
58335, 0.0003591425313376732, 7.9618794111583e-05, 2.16542118724904e-05, 0.000391209
1825367341, 0.0003366899003069471, -0.0007028423756221421, -0.000555066011061375, -
0.003805501872764046, -0.004814378904061813, 0.002977261042104989, 0.008910950446988
06, 0.008366001043194241, 0.006410574020412633, 0.0028547171100597817, 0.00100365753

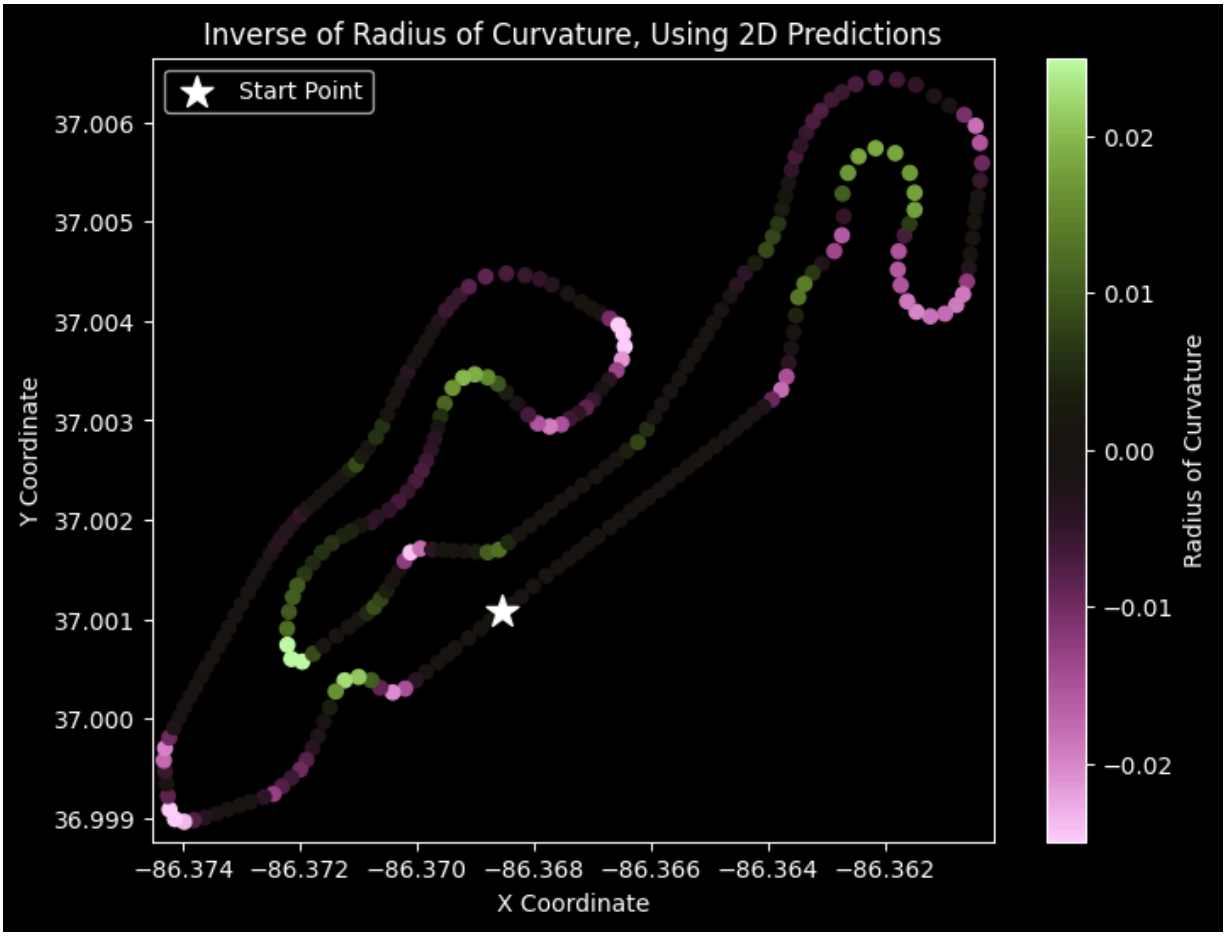
```
20589633, -0.00037029535046739483, -0.004897224720277334, -0.006642294747426724, -0.
004797179351200363, -0.005445852282269572, -0.007307659274926706, -0.007475080920338
971, -0.00662297773158883, -0.005752635951898043, -0.006948076693932992, -0.00752844
5346630799, -0.006367036957014123, -0.00503611319301257, -0.0022624589353555185, -0.
0017068781026176208, -0.010665383490019916, -0.018139477673595042, -0.01559234122049
8202, -0.009708596730716938, -0.005578329817479912, -0.0007846847332923172, 0.000872
0964224366604, 0.0007589045423913788, -8.326945393197489e-05, -7.152840668990327e-0
5, -0.003473295373611589, -0.012427708675629806, -0.018790555192388907, -0.019106127
920062842, -0.017823423884625576, -0.018434035086039868, -0.020030336203015475, -0.0
18736251969977736, -0.015557536246111742, -0.016185192888943097, -0.015416387490819
7, -0.006854570780931154, 0.007377466258863164, 0.01790256587256852, 0.0178192441116
8935, 0.017375879014013638, 0.018345111937897025, 0.018610681388772105, 0.0179827110
48118695, 0.01684067128667242, 0.010722513011284416, -0.004967793472694951, -0.01611
0810852779277, -0.0138429052755666, -0.0026939287370878772, 0.006618365049955982, 0.
014092856770176226, 0.013472944876994443, 0.0042570327531942, -0.001223722561647761
7, -0.0007296889848569075, -0.004177197380907719, -0.014939360437847557, -0.01787488
431833558, -0.00940323032458375, -0.002135779365853173, 0.0001375775690997799, -0.00
031216653698923824, -0.000614395419602564, -0.0003972275517051777, 5.548898587490303
e-05, 2.50904963626677e-05, -0.00014105921878708629, 0.00018306541184922434, 0.00011
1472361206557, -0.0005712623117563767, -0.0011424224539213041, -0.000110250234743337
79, 0.0010769790766157067, 0.0002797789339890044, -0.0007289645654788712, -9.9297944
77615497e-05, 0.0008273543490389876, 0.0003809217866092804, -0.0006155994243415151,
-0.0005839268115951013, 0.0002864881047732589, 0.0010669537924977727, 0.000626089247
5680676, -0.0004704293692957141, -0.0007506187416346773]
```

We can plot this on a graph. We expect to see the corners and straights colored differently!

In [20]:

```python
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection

# Separating the coordinates into x and y components
x_coords = [coord[1] for coord in coords]
y_coords = [coord[0] for coord in coords]

# Create an array of points from the coordinates
points = np.array([x_coords, y_coords]).T.reshape(-1, 1, 2)

# Create a plot
plt.figure(figsize=(8, 6))
plt.scatter(x_coords, y_coords, c= radius_of_curvature)
plt.gca().get_yaxis().get_major_formatter().set_useOffset(False)

# Add a color bar to show the velocity scale
plt.colorbar( label='Radius of Curvature')
plt.clim(-0.025, 0.025)
plt.set_cmap("vanimo")
# Add labels, grid, and title
plt.xlabel('X Coordinate')
plt.ylabel('Y Coordinate')
plt.title("Inverse of Radius of Curvature, Using 2D Predictions")



# Adjust the axis limits to fit the track
plt.xlim(min(x_coords) - 0.0002, max(x_coords) + 0.0002)
plt.ylim(min(y_coords) - 0.0002, max(y_coords) + 0.0002)
```

```
# Optionally, highlight the starting point
plt.scatter(x_coords[0], y_coords[0], color='white', s=200, marker='*', label='Star

# Add a legend and show the plot
plt.legend()
plt.show()
```

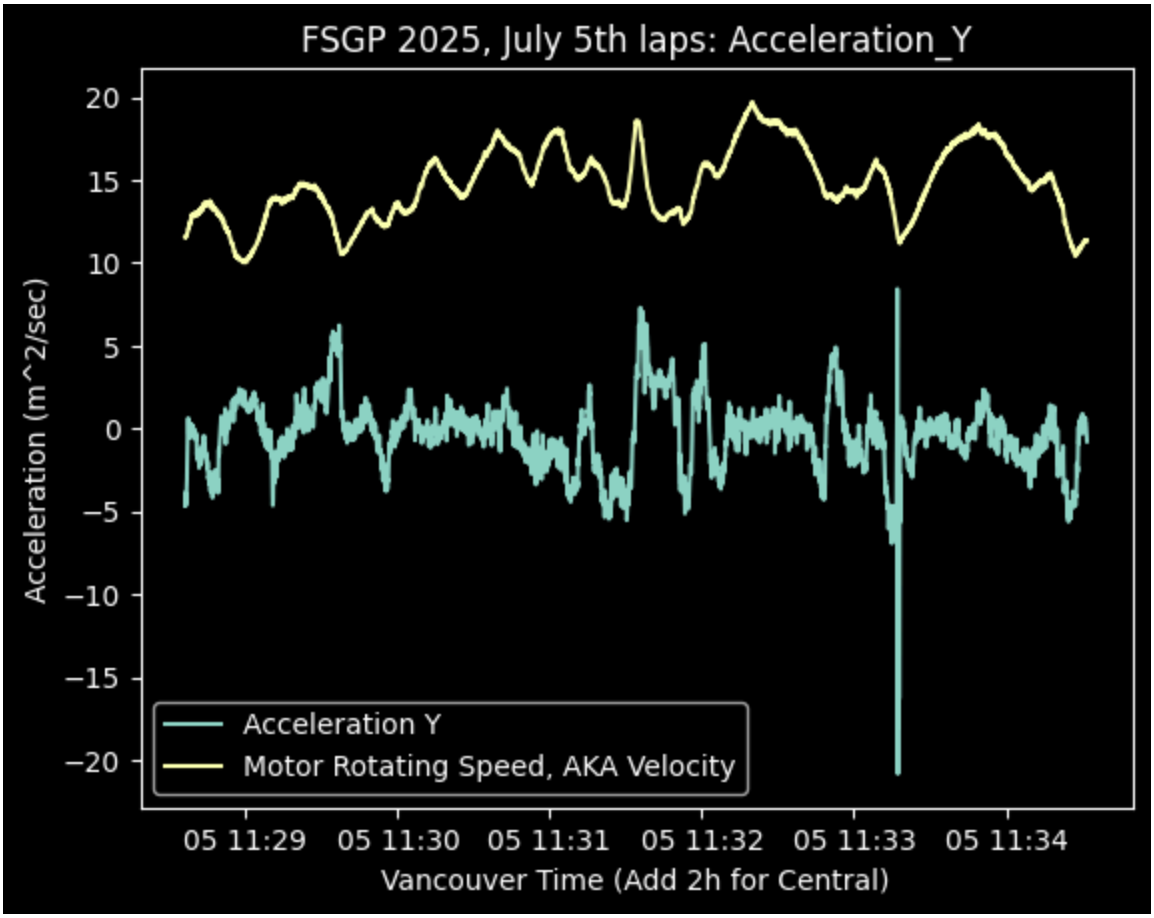


Working with our acceleration data

We want to generate the same graph using sensor data to find radii instead of GPS points. Here I run a sanity check to see our data query is working!

In an earlier project, Jonah identified that Acceleration_Y was the centripetal acceleration. See his acceleratiom_analysis project for more details. We're going to use this assumption for the rest of the project

In [21]:
```
plt.plot(y.datetime_x_axis, y, label = "Acceleration Y")
plt.plot(motor.datetime_x_axis, motor, label = "Motor Rotating Speed, AKA Velocity"
plt.ylabel("Acceleration (m^2/sec)")
plt.xlabel("Vancouver Time (Add 2h for Central)")
plt.title("FSGP 2025, July 5th laps: Acceleration_Y")
plt.legend()
plt.show()
```



The data from the sensor is noisy but generally sensical. The following function is a botched attempt at finding the radii using data queried from the database. Later, I decide it's better to do this using just Acceleration_Y as centripetal force, but this is another theoretically valid approach.

In [22]:
```
def calculate_radius_from_scaled_velocity(v_raw, ax_raw, ay_raw, dt):
    # 1. Force conversion to standard numpy arrays immediately
    # This bypasses the custom TimeSeries __getitem__ logic
    v_scaled = np.asarray(v_raw)
    ax = np.asarray(ax_raw)
    ay = np.asarray(ay_raw)

    # 3. Calculate Tangential Acceleration (dv/dt)
    # Using v directly (assuming it is already m/s based on your previous 'scaled'
    # If it's still in RPM, multiply by (2 * np.pi / 60) * radius here
    a_tangential = np.gradient(v_raw, dt)

    # 4. Get Total Acceleration Magnitude squared
    a_total_sq = ax**2 + ay**2

    # 5. Extract the Centripetal (Turning) Acceleration
    # a_centripetal^2 = a_total^2 - a_tangential^2
    a_centripetal_sq = np.clip(a_total_sq - a_tangential**2, 0, None)
    a_centripetal = np.sqrt(a_centripetal_sq)

    # 6. Calculate Radius (R = v^2 / a_centripetal)
    radius = np.full_like(v_raw, np.inf)
```

```
    # Avoid division by zero
    mask = (a_centripetal > 1e-4) & (v_raw > 0.1)
    radius[mask] = (v_raw[mask]**2) / a_centripetal[mask]

    return radius
```

In [23]:
```
calculated_radius = calculate_radius_from_scaled_velocity(motor, x, y, 0.1)

radius_with_xy = 1/calculated_radius
```

In [24]:
```
print(radius_with_xy)
```

```
[0.07897489 0.07379561 0.07756482 ... 0.07881478 0.07677762 0.0713407 ]
```

The following function is much better at calculating the radius and generally seems to use the sensor data more correctly.

In [25]:
```
def calculate_radius_direct(v, a_centripetal):
    """
    Calculates radius of curvature directly from velocity and centripetal accelerat

    Parameters:
    -----------
    v : np.ndarray
        Linear velocity (speed) in m/s.
    a_centripetal : np.ndarray
        The lateral/centripetal component of acceleration in m/s^2.
    """
    # Initialize with infinity (represents driving straight)
    radius = np.full_like(v, np.inf, dtype=float)

    # Create a mask to avoid DivisionByZero or invalid math
    # 1. Must be moving (v > 0)
    # 2. Must be turning (a_c > 0)
    mask = (np.abs(a_centripetal) > 1e-5) & (np.abs(v) > 1e-5)

    # R = v^2 / a_c
    radius[mask] = (v[mask]**2) / a_centripetal[mask]

    return radius
```

In [26]:
```
centri_radius= calculate_radius_direct(motor, y)
calculate_radius_yonly = 1/centri_radius
print(calculate_radius_yonly)
len(calculate_radius_yonly)
```

```
[-0.03483339 -0.02870119 -0.03252474 ... -0.00175272 -0.00266378
 -0.0060383 ]
```

Out[26]: 3559

We now have a radius of curvature for each point of sensor data. But to graph this, we need it in relation to our indexed race-track points. We're going to import GIS packages to work

with our coordinates and find the length of the lap. Borrowed this method from Miguel's previous work.

In [27]:
```
from physics.environment.gis.gis import GIS
from physics.environment.gis.gis import calculate_path_distances

# Defining dictionary "route_data"

route_data = {
    "path" : np.array(coords),
    "elevations" : np.zeros(len(coords)),
    "time_zones" : np.zeros(len(coords)),
    "num_unique_coords" : (len(coords) - 1) }

# Creating GIS object

starting_coords = [37.00107373, -86.36854755]

gis = GIS(route_data, starting_coords, current_coord = starting_coords)

lap_length = np.cumsum(calculate_path_distances(gis.path[:gis.num_unique_coords]))[
```

We find the distance we have travelled so far, and make a list of distances travelled as time goes on.

In [28]:
```
def distance_so_far(v_ts, start_time, end_time, dt=0.1):
    return v_ts*v_ts.period
```

In [29]:
```
distance_list = distance_so_far(motor, lap_start_3_3, lap_stop_3_4, dt=0.1)
```

We give our list of distances to the return_coords function, which returns the nearest Google Earth coordinate for easy graphing.

In [30]:
```
def return_coords(distances:list):
    total_lap_distance = sum(distances)
    total_track_length = 5033.62413471853
    index = gis._python_calculate_closest_gis_indices(distances*(total_track_length
    return route_data.get("path")[index], index
```

We can run this function here.

In [31]:
```
coords_calc, index = return_coords(distance_list)
len(calculate_radius_yonly)
```

```
Calculating closest GIS indices: 100%|██████████| 3559/3559 [00:00<00:00, 1313811.65
it/s]
```

Out[31]: 3559

Because there are thousands of sensor points and only 288 Google Earth points, we can group sensor data by nearest indexed point and summarize the mean radius. This has an added benefit of smoothing our data for better graphing.

In [32]:
```
mean_radius_per_index = (
    pd.Series(calculate_radius_yonly)
      .groupby(index)
      .mean()
)
print(mean_radius_per_index)
```

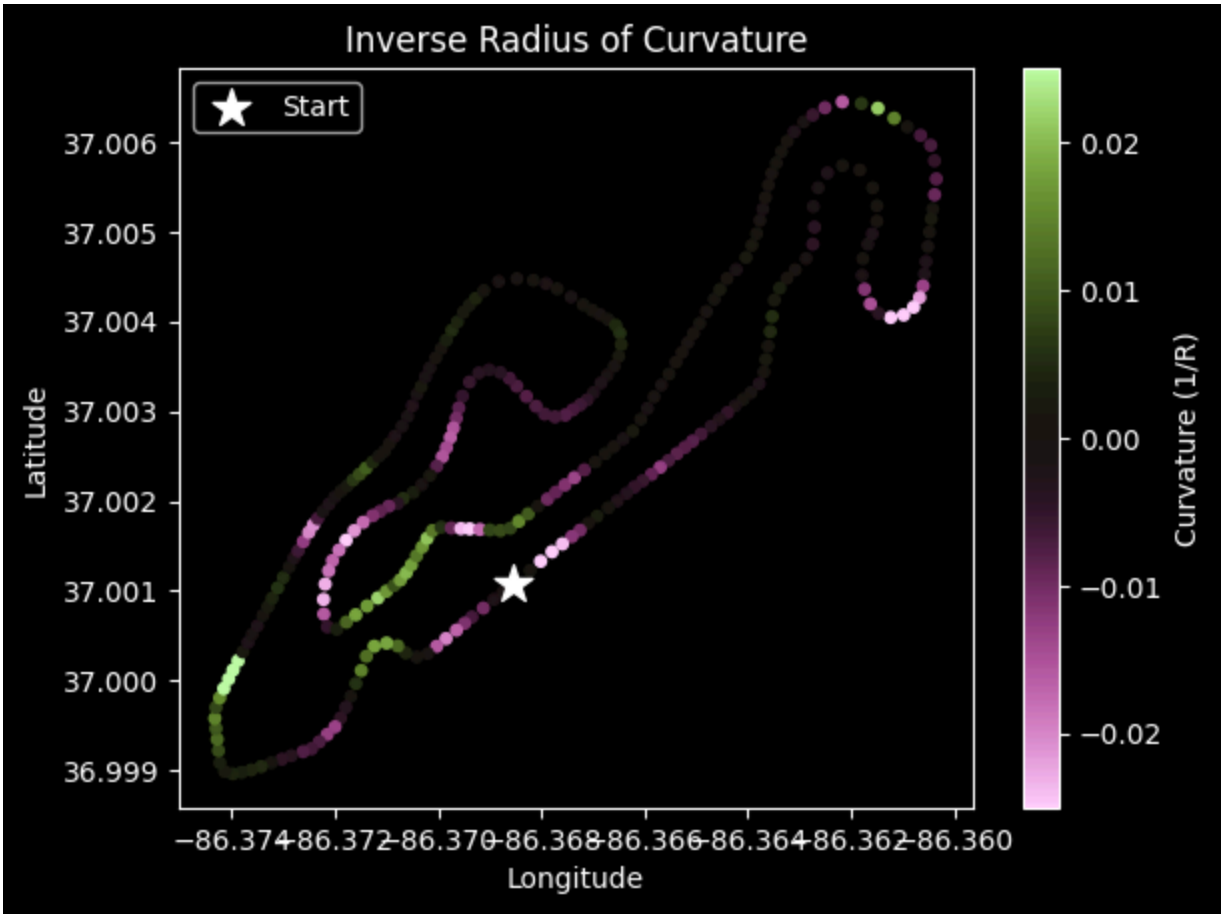
```
0     -0.017007
1     -0.000481
2     -0.002090
3     -0.010024
4     -0.006829
         ...
284   -0.011722
285   -0.024066
286   -0.037663
287   -0.032725
288    0.001187
Length: 288, dtype: float64
```

We can now try and generate the same graph from earlier but using the sesnor data instead of points on the Google Earth map.

In [33]:
```
new_coords = [(round(x, 5), round(y, 5)) for x, y in coords]
unique_coords = list(dict.fromkeys(new_coords))
len(unique_coords)
```

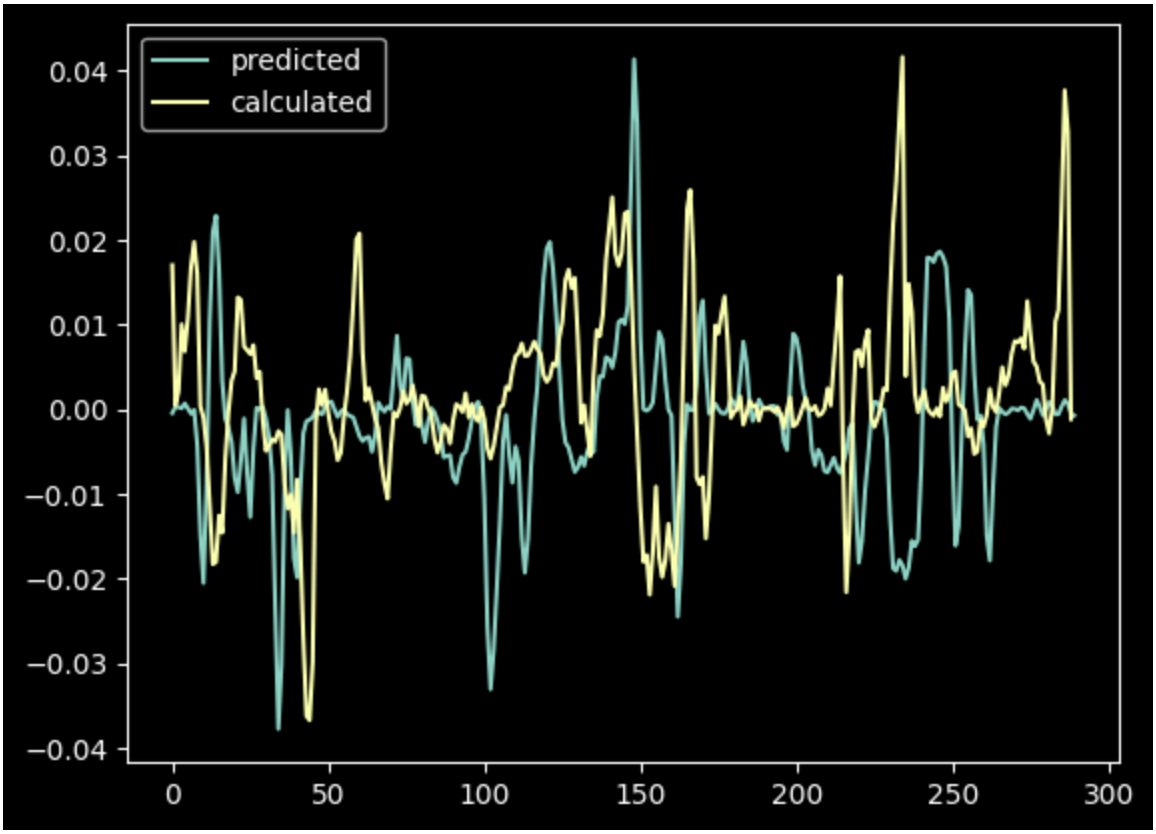
Out[33]: 289

In [34]:
```
# 1. Align coordinates with the data indices
# mean_radius_per_index.index tells us exactly which track points have data
valid_idx = mean_radius_per_index.index
x_plot = np.array(x_coords)[valid_idx]
y_plot = np.array(y_coords)[valid_idx]
colors = mean_radius_per_index.values



sc = plt.scatter(x_plot, y_plot, c=colors, cmap='vanimo', s=15)

# Formatting
plt.colorbar(sc, label="Curvature (1/R)")
plt.clim(-0.025, 0.025) # Keeps the scale consistent
plt.gca().get_yaxis().get_major_formatter().set_useOffset(False)

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Inverse Radius of Curvature")

# Highlight start
plt.scatter(x_plot[0], y_plot[0], color='white', marker='*', s=200, label='Start')

plt.legend()
plt.show()
```



Here, we compare the results in a line graph, to see major differences.

In [35]:
```
plt.plot(radius_of_curvature, label = "predicted")
plt.plot(-mean_radius_per_index, label = "calculated")
plt.legend()
plt.show()
```



Using Elevation Data to Fine-tune Prediction Model

We're seeing quite a stark difference between our predicted and actual, especially in between indexed points 50-100 and 150-200. I was wondering if this might be because of elevation, which we didn't consider in our predicted model. Using a Google Maps API, we can find the elevation at the coordinate points and integrate that into our model. The addition of 3D spatial data may explain the discrepency in magnitude we see in our trouble areas.

In [36]:
```
load_dotenv()

def _calculate_path_elevations(coords):
    """

    Returns the elevations of every coordinate in the array of coordinates passed i
    See Error Message Interpretations: https://developers.google.com/maps/documenta

    :param np.ndarray coords: A NumPy array [n][latitude, longitude]
    :returns: A NumPy array [n][elevation] in metres
    :rtype: np.ndarray

    """

    # construct URL
    url_head = "https://maps.googleapis.com/maps/api/elevation/json?locations="

    location_strings = []
```

```
    locations = ""

    for coord in coords:
        locations = locations + f"{coord[0]},{coord[1]}|"

        if len(locations) > 8000:
            location_strings.append(locations[:-1])
            locations = ""

    if len(locations) != 0:
        location_strings.append(locations[:-1])

    url_tail = f"&key={os.environ['GOOGLE_MAPS_API_KEY']}"

    # Get elevations
    elevations = np.zeros(len(coords))

    i = 0
    with tqdm(
        total=len(location_strings), file=sys.stdout, desc="Acquiring Elevation Dat
    ) as pbar:
        for location_string in location_strings:
            url = url_head + location_string + url_tail

            r = requests.get(url)
            response = json.loads(r.text)
            pbar.update(1)

            if response["status"] == "OK":
                for result in response["results"]:
                    elevations[i] = result["elevation"]
                    i = i + 1

            elif response["status"] == "INVALID_REQUEST":
                sys.stderr.write("Error: Request was invalid\n")

            elif response["status"] == "OVER_DAILY_LIMIT":
                sys.stderr.write(
                    "Error: Possible causes - API key is missing or invalid, billin
                    " a self-imposed usage cap has been exceeded, or the provided p
                    " valid. \n"
                )

            elif response["status"] == "OVER_QUERY_LIMIT":
                sys.stderr.write("Error: Requester has exceeded quota\n")

            elif response["status"] == "REQUEST_DENIED":
                sys.stderr.write("Error: API could not complete the request\n")

    return elevations
```

In [37]:
```
elevations = _calculate_path_elevations(coords)
```

```
Acquiring Elevation Data: 100%|██████████| 1/1 [00:00<00:00,  6.40it/s]
```

We can now create a more involved, three-dimensional predicted model for our curvature.

In [38]:
```
def predicted_with_elevation(coords, elevations, step=2, elevation_weight=3.0):
    """
    Calculates SIGNED 3D curvature with amplified elevation influence.
    Positive = Right Turn, Negative = Left Turn.

    :param elevation_weight: Multiplier for elevation changes (default 3.0)
                             Higher values make slopes affect curvature more
    """
    if len(coords) != len(elevations):
        raise ValueError("The number of coordinates must match the number of elevat

    # 1. Project to Local X/Y/Z with weighted elevation
    center_lat = coords[0][0]
    WGS84_LATITUDE_BASE = 111_132.954
    WGS84_LATITUDE_COS_COEFF = 559.822
    WGS84_LONGITUDE_BASE = 111_412.84

    meters_per_deg_lat = (WGS84_LATITUDE_BASE - WGS84_LATITUDE_COS_COEFF * math.cos
    meters_per_deg_lon = (WGS84_LONGITUDE_BASE * math.cos(center_lat_rad))

    xyz_points = []
    for i in range(len(coords)):
        lat, lon = coords[i]
        xyz_points.append((
            lon * meters_per_deg_lon,
            lat * meters_per_deg_lat,
            elevations[i] * elevation_weight  # Amplify vertical component
        ))

    curvature = []
    n = len(xyz_points)

    for i in range(n):
        p1 = xyz_points[(i - step) % n]
        p2 = xyz_points[i]
        p3 = xyz_points[(i + step) % n]

        # --- 3D Magnitude Calculation ---
        a = math.dist(p1, p2)
        b = math.dist(p2, p3)
        c = math.dist(p3, p1)

        s = (a + b + c) / 2
        area_sq = s * (s - a) * (s - b) * (s - c)
        area = math.sqrt(max(0, area_sq))
        denom = a * b * c

        if area < 1e-6 or denom < 1e-6:
            k = 0
        else:
            k = (4 * area) / denom

        # --- Direction (Sign) Calculation ---
```

```
        v1 = (p2[0] - p1[0], p2[1] - p1[1])
        v2 = (p3[0] - p2[0], p3[1] - p2[1])
        cross_product = (v1[0] * v2[1]) - (v1[1] * v2[0])

        if cross_product < 0:
            k = -k  # Left turn

        curvature.append(k)

    return curvature
```

In [39]:
```
three_dim_predict = predicted_with_elevation(coords, elevations, step=4, elevation_
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[39], line 1
----> 1 three_dim_predict = predicted_with_elevation(coords, elevations, step=4, ele
vation_weight=20)

Cell In[38], line 18, in predicted_with_elevation(coords, elevations, step, elevatio
n_weight)
     15 WGS84_LATITUDE_COS_COEFF = 559.822
     16 WGS84_LONGITUDE_BASE = 111_412.84
---> 18 meters_per_deg_lat = (WGS84_LATITUDE_BASE - WGS84_LATITUDE_COS_COEFF * math.
cos(2 * center_lat_rad))
     19 meters_per_deg_lon = (WGS84_LONGITUDE_BASE * math.cos(center_lat_rad))
     21 xyz_points = []

NameError: name 'center_lat_rad' is not defined
```

We can now plot this, to see if it looks more similar to what we calculated.

In [ ]:
```
# Separating the coordinates into x and y components
x_coords = [coord[1] for coord in coords]
y_coords = [coord[0] for coord in coords]

# Create an array of points from the coordinates
points = np.array([x_coords, y_coords]).T.reshape(-1, 1, 2)

# Create a plot
plt.figure(figsize=(8, 6))
plt.scatter(x_coords, y_coords, c= three_dim_predict)
plt.gca().get_yaxis().get_major_formatter().set_useOffset(False)

# Add a color bar to show the velocity scale
plt.colorbar( label='Radius of Curvature')
plt.clim(-0.025, 0.025)
plt.set_cmap("berlin")
# Add labels, grid, and title
plt.xlabel('X Coordinate')
plt.ylabel('Y Coordinate')
plt.title("Predicted Inverse of Radius of Curvature, using Elevation Data")
```

```
# Adjust the axis limits to fit the track
plt.xlim(min(x_coords) - 0.0002, max(x_coords) + 0.0002)
plt.ylim(min(y_coords) - 0.0002, max(y_coords) + 0.0002)

# Optionally, highlight the starting point
plt.scatter(x_coords[0], y_coords[0], color='white', s=200, marker='*', label='Star

# Add a legend and show the plot
plt.legend()
plt.show()
```

Overall, this seems to have reduced the issues of mismatched amplitude slightly. We can investigate this more with a line graph.

In [ ]:
```
plt.plot(three_dim_predict, label = "Predicted with 3D data")
plt.plot(-mean_radius_per_index, label = "Calculated")
plt.legend()
plt.show()
```

And we can compare the 2D and 3D models as well.

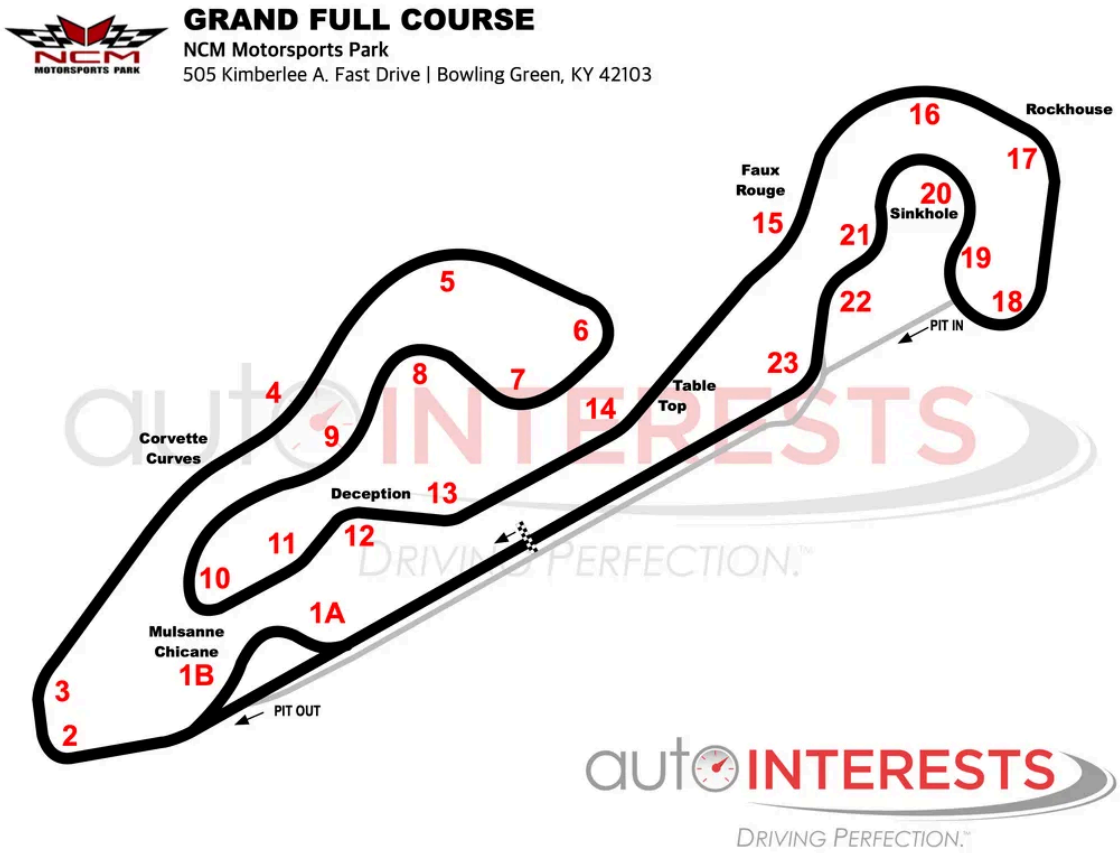
In [ ]:
```
plt.plot(radius_of_curvature, label = "Predicted with 2D data", color = "pink", alp
plt.plot(three_dim_predict, label = "Predicted with 3D data", color = "lightblue",
plt.plot(-mean_radius_per_index, label = "Calculated", color = "white", alpha = 0.5
plt.legend()
plt.show()
```

In [ ]:
```
plt.plot(three_dim_predict, label = "Predicted with 3D data")
plt.plot(radius_of_curvature, label = "Predicted with 2D Data", color = "pink", alp
plt.legend()
plt.show()
```

Investigating Our Model in Context

The question we tried to answer was if we could localize our data. The answer? Maybe!

Our model seems to work generally except for certain, specific spots. What happened there is a mystery. Aarjav shared a reflection written during the race that specifies some confounding variables we might want to consider. Day 3 was relying on a borrowed motor from UofT's team and the car could only drive very slow. The later discontinuity in amplitude between points 150-200 reflect where our team decided not to turn through the "sinkhole" (around point 20 on the image below) and took an alternate route instead. This change in route also would've made the racetrack shorter, which makes our earlier calculation using the track length off.



But this still doesn't explain the earlier zone where we expect to see curvature and don't, as well as other minute errors along the path. Generally, this project leads me to believe that we would benefit from a consistent time-stamped video during the race (from a Go-Pro, maybe) so we don't have to wonder about what happened. This would help our team with post-morten data analysis, and other teams shared similar interest in our DR day!

Generally, I've shown we can localize our data. Doing so with another set of variables and with less assumptions than I did with this specific project may yield better results.